The group lasso in EXTRACT

what is fitted, how, and how it enters training (branch `main`)

EXTRACT project notes

October 22, 2026

1 The idea in one paragraph

Every row of the data is the response of one (perturbation, context) pair: the pair's full-data estimate or one of its 100-cell pseudobulks. We write each row as a combination of the $d = 20$ programs, $\mathbf{x} \approx \mathbf{z}B$. Without sparsity, a row may use all 20 programs. The group lasso asks for the opposite: each *pair* should use only a few programs, and all rows of the same pair must use the *same* few. How strongly each chosen program is used may still differ between the rows of the pair. A perturbation can use different programs in different contexts, because each (perturbation, context) pair makes its own choice.

2 What is fitted

Notation.

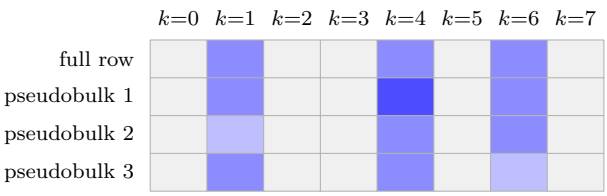
- B: the $d \times G$ loadings, one row $\mathbf{b}_k$ per program ($G = 5{,}504$ genes, no mitochondrial genes). In the fixed mode $B = AV$, with V the PCA of the training pair means (fixed) and A learned. Every row of B is kept at unit length.
- A pair g has n_g rows $\mathbf{x}_1, \ldots, \mathbf{x}_{n_g}$ (full row + pseudobulks; all rows of a pair are exchangeable, the model is not told which is which or how many cells they have).
- Z_g: the $n_g \times d$ activities of the pair's rows; row r is $\mathbf{z}_r$, column k, written $Z_g[:, k]$, is program k's activity in each of the pair's rows.
- $\mathbf{m}_r$: a 0/1 mask over genes that removes the row's own target gene (its knockdown would dominate).

The objective, for one pair.

$$\min_{Z_g} \underbrace{\tfrac{1}{2} \sum_{r=1}^{n_g} \left\| \mathbf{m}_r \odot (\mathbf{x}_r - \mathbf{z}_r B) \right\|^2}_{\text{reconstruction error}} + \underbrace{\lambda \sqrt{n_g} \sum_{k=1}^{d} \left\| Z_g[:, k] \right\|_2}_{\text{group penalty}} \tag{1}$$

- **The penalty acts on columns.** $\|Z_g[:, k]\|_2$ is the length of program k's activity vector across the pair's rows. A group penalty on such lengths sets whole columns exactly to zero: program k is then off for *every* row of the pair. A column that survives can take any values, so the rows of the pair may use a chosen program with different strengths.
- $\sqrt{n_g}$ makes the penalty comparable between pairs with few and many rows (a column's length grows like $\sqrt{n_g}$).
- λ sets how few programs a pair uses: larger λ, fewer programs.

- **Unit-length rows of B are required.** Otherwise a program could be scaled up, its activities scaled down by the same factor, and the penalty would shrink for free.



one pair: 3 programs used (blue columns), the same for all its rows; strengths differ between rows; all other columns exactly 0

Figure 1: What the group lasso produces for one pair (8 of the 20 programs shown).

3 How it is solved

Equation (1) has no closed form; it is solved by **FISTA** (proximal gradient with momentum), separately for every pair, all pairs of a batch in parallel:

1. **Start** at the least-squares activities (the masked projection of each row onto B).
2. **Gradient step** on the reconstruction error, step size $\eta = 1/L$ with L the largest eigenvalue of $BB^\top$.
3. **Group soft-thresholding** of each column:

$$Z_g[:,k] \;\leftarrow\; Z_g[:,k] \cdot \max\Big(0,\; 1 - \frac{\eta\,\lambda\sqrt{n_g}}{\|Z_g[:,k]\|_2}\Big),$$

   which shrinks every column and sets the small ones exactly to zero.
4. **Momentum** (the FISTA extrapolation), and repeat; 30 steps.

The programs whose column is non-zero at the end are the pair's **selected programs**. On the real data, 30 steps select the same programs as 150 steps in 99.7–99.9% of cases. The steps are ordinary differentiable operations, so gradients flow back to B through them (the starting point is held fixed).

One fit. The same activities are used everywhere: in the reconstruction objective below and by the discriminator.

4 How it enters training

Loss. With the group lasso and only label swaps as false examples (the chosen setting; no shuffled rows), the training loss is

$$\text{total} \;=\; L_{\text{disc}} + \alpha\, L_{\text{lasso}}, \qquad L_{\text{lasso}} \;=\; \frac{1}{n}\sum_{\text{rows}} \frac{\|\mathbf{m}_r \odot (\mathbf{x}_r - \mathbf{z}_r B)\|^2}{G} + \frac{2\lambda}{nG} \sum_{\text{pairs } g} \sqrt{n_g} \sum_{k} \|Z_g[:,k]\|_2, \tag{2}$$

with n the number of rows in the batch and Z the lasso activities. L_{lasso} is exactly equation (1), summed over the batch's pairs and divided by $nG/2$, so it is on the same per-row, per-gene scale as before. L_{disc} is the binary discriminator loss on the same activities ($\alpha = 700$).

λ appears twice. In the solve (which programs each pair selects) and in the loss (so the gradient on A feels the penalty: axes on which pairs need fewer programs become cheaper).

Per step.

1. A batch is made of *whole pairs* (all rows of a pair together), about 1,024 rows.
2. Solve equation (1) for every pair of the batch with the current B.
3. Compute L_{disc} and L_{lasso} on these activities; backpropagate; one optimiser step on A and the label network.
4. Rescale the rows of $B = AV$ to unit length.

Validation and inference also need whole pairs: each row must be solved together with the other rows of its pair.

5 Choosing λ

The natural scale of λ is the typical size of a program's activity in a pair. We measured it as the median, over pairs and programs, of $\|Z_g[:,k]\|_2/\sqrt{n_g}$ for the least-squares activities on the joint diagonalisation axes: **0.0272** at $d = 20$. The grid was $\lambda \in \{1, 2, 4, 7.4, 14.7\} \times 0.0272$.

λ	0.0272	0.0544	0.1088	0.2	0.4
programs per pair (validation)	6.2	3.9	2.2	1.3	0.6
validation AUC	0.808	0.763	0.726	0.686	0.645
reconstruction explained	51.7%	51.5%	51.2%	50.6%	49.4%

Table 1: Chosen setting (4-statistics head with normaliser, label swaps only, JD start, no-MT data), mean over 3 seeds. Fewer programs per pair cost discrimination but barely any reconstruction: a pair's response is carried by very few directions.

6 Why it helps choose the axes

Reconstruction alone cannot choose the axes inside V: in the fixed mode every rotation of the axes fits the data equally well. The group penalty can. If the true programs are the axes, a pair's response lies on a few of them and needs few non-zero columns. Rotate the axes, and the same response is spread over many programs: more non-zero columns, a larger penalty. So the lasso objective prefers axes on which pairs are sparse.

We checked this without any training, on 3,000 training pairs ($d = 20$, $\lambda = 2\times$ scale, data with mitochondrial genes):

axes	lasso objective	error	programs per pair
ICA	1.5267	1.4876	2.77
PCA (V itself)	1.5296	1.4848	2.27
joint diagonalisation (JD)	1.5305	1.4884	2.83
JD rotated, $t = 0.25$	1.5312	1.4885	2.89
JD rotated, $t = 0.5$	1.5326	1.4887	2.94
JD rotated, $t = 1$	1.5357	1.4891	3.08
JD rotated, $t = 2$	1.5399	1.4895	3.18

Table 2: The objective rises steadily as the axes are rotated away from JD, mostly through the penalty (more programs per pair). The preference is real but small, which is why, in training, the discriminator can still move the axes.

7 What it does not do

- It does not make the *gene* loadings sparse; it makes each pair use few programs. Sparsity over genes is a separate prior on B (under discussion).
- It does not use labels, cell counts or the identity of the full row: the grouping only says which rows belong to the same pair.
- It does not force a perturbation to use the same programs in every context.